🔷 BLOC 1 — DIAGNOSTIC

👉 *Objectif : comprendre l'état de la machine avant toute action*

1.1 Identifier l'environnement (OS / noyau)

Commandes

```
cat /etc/debian_version
lsb_release -a
uname -r
```

Pourquoi on commence par là

- On doit **savoir exactement sur quel système on travaille**.
- Certaines commandes, fichiers ou comportements **dépendent de la version Debian et du noyau**.

Ce que ça implique / impact

- Aucune modification système.
- Simple lecture d'informations.

Ce que ça génère

- Version Debian
- Codename (bookworm)
- Version du noyau Linux

➡️ **Utilité TSSR** : documenter le contexte, expliquer pourquoi une config est compatible.

26-01-21-COMMANDE AVANCE - SCRI…

1.2 Vérifier les ressources système (CPU / RAM / disque)

Commandes

```
cat /proc/cpuinfo
free -h
df -h
uptime
```

Pourquoi on le fait

- Vérifier que la machine **peut supporter ce qu'on va faire** (scripts, services, réseau).
- Détecter un problème avant qu'il bloque le TP (plus de disque, RAM saturée…).

Ce que ça implique

- Lecture uniquement.
- Aucun impact sur la machine.

Ce que ça génère

- Nombre de cœurs CPU
- RAM disponible
- Espace disque restant
- Charge système

➡️ **Logique TSSR** : on ne configure jamais "à l'aveugle".

26-01-21-COMMANDE AVANCE - SCRI…

26-01-22- SCRIPT

1.3 Vérifier le réseau (IP / interface / DNS)

Commandes

```
ip a
ip -br a
ip l
cat /etc/resolv.conf
```

Pourquoi

- **Sans réseau valide**, tout le reste peut échouer (SSH, DNS, scripts, mises à jour).
- On confirme :
  - Interface UP
  - IP correcte
  - DNS fonctionnel

Ce que ça implique

- Lecture uniquement.

Ce que ça génère

- IP de l'interface `enp0s3`
- État UP/DOWN
- Serveur DNS utilisé

➡️ **Impact réel** : si DNS ou IP est faux, beaucoup d'actions échouent ensuite.

26-01-21-COMMANDE AVANCE - SCRI…

🔷 BLOC 2 — EXTRACTION

👉 *Objectif : transformer l'information brute en données exploitables*

2.1 Comprendre et manipuler les sorties de commandes

Commandes

```
ls -l
ls -l > liste.txt
ls -l >> liste.txt
ls -l | grep scripts
```

Pourquoi

- On apprend à :
  - **capturer une sortie**
  - **filtrer une information**
  - **préparer une automatisation**

Ce que ça implique

- Création ou modification de fichiers texte.
- Aucun impact système critique.

Ce que ça génère

- Fichiers de sortie exploitables (`liste.txt`)
- Résultats filtrés sans fichier intermédiaire

➡️ **Fondation du scripting Linux**.

26-01-21-COMMANDE AVANCE - SCRI…

2.2 Extraire des champs avec `cut`

Commande type

```
cut -d':' -f 1 users.txt
```

Pourquoi

- Les données sont **structurées** (séparateur `:`).
- On transforme une ligne en **variable exploitable** (login, groupe, mot de passe).

Ce que ça implique

- Lecture d'un fichier texte.
- Aucun impact système.

Ce que ça génère

- Une valeur précise par ligne.

➡️ **Utilité clé pour les scripts d'utilisateurs**.

26-01-21-COMMANDE AVANCE - SCRI…

2.3 Quand `cut` ne suffit pas → `awk`

Exemple réseau (du PDF)

```
ip -br a | grep enp0s3 | awk '{print $3}'
```

Puis :

```
cut -d'/' -f 1
```

Pourquoi

- Certaines sorties sont **moins régulières**.
- `awk` permet une extraction plus fine.

Ce que ça implique

- Lecture + traitement mémoire uniquement.

Ce que ça génère

- IP propre (sans `/24`) utilisable dans un script.

➡️ **Automatisation fiable**.

26-01-22- SCRIPT

🔷 BLOC 3 — AUTOMATISATION

👉 *Objectif : appliquer des actions système de façon contrôlée et répétable*

3.1 Structure d'un script

Base

```
#!/bin/bash
# commentaire
```

Pourquoi

- `#!/bin/bash` indique **quel interpréteur utiliser**.
- Les commentaires expliquent le rôle du script.

Impact

- Aucun impact tant que le script n'est pas exécuté.

26-01-21-COMMANDE AVANCE - SCRI…

3.2 Sécuriser le scénario (tests + conditions)

Exemple : vérifier un fichier

```
if ! [[ -f ${fichier} ]]
then
  exit
fi
```

Pourquoi

- On évite un script qui **continue sans données valides**.
- C'est une **bonne pratique professionnelle**.

Impact

- Le script s'arrête proprement si condition non remplie.

26-01-22- SCRIPT

3.3 Boucle de traitement (import utilisateurs)

Boucle

```
for ligne in $(cat users.txt)
do
  login=$(echo ${ligne} | cut -d':' -f 1)
done
```

Pourquoi

- Chaque ligne représente **une action à automatiser**.
- On applique le même traitement à plusieurs utilisateurs.

Impact système

- Aucune modification tant que les commandes système ne sont pas appelées.

26-01-22- SCRIPT

3.4 Actions système réelles (création utilisateurs)

Commandes

```
useradd login --home-dir /home/login --create-home
echo login:password | chpasswd
usermod -aG sudo login
```

Pourquoi

- `useradd` : création non interactive (scriptable).
- `chpasswd` : définir le mot de passe.
- `usermod` : droits sudo si nécessaire.

Impact réel

- Écriture dans :
  - `/etc/passwd`
  - `/etc/shadow`
  - `/etc/group`
- Création du `/home/login`

➡️ **Action critique → doit être maîtrisée et expliquée**.

26-01-22- SCRIPT

3.5 Gestion des erreurs (`$?`)

Exemple

```
if [[ $? -ne 0 ]]
then
  exit
fi
```

Pourquoi

- On vérifie que l'action précédente a réussi.
- On évite un système partiellement configuré.

Impact

- Script plus robuste, plus professionnel.

26-01-22- SCRIPT

3.6 Automatisation réseau + hostname

Commandes clés

```
sed -i.bak "/^address/caddress ${ipv4}" /etc/network/interfaces
sed -i "/^netmask/cnetmask ${mask}" /etc/network/interfaces
sed -i "/^gateway/cgateway ${gw}" /etc/network/interfaces
hostnamectl set-hostname nouveau_nom
reboot
```

Pourquoi

- Modifier IP, passerelle et nom machine **de façon cohérente**.
- Le `.bak` permet un **retour arrière**.

Impact réel

- Modification réseau persistante.
- Perte de connexion possible après reboot (normal).
- Nouvelle identité machine.

26-01-21-COMMANDE AVANCE - SCRI…

26-01-22- SCRIPT

🧠 Synthèse mentale (très importante)

- **Diagnostic** → je comprends l'état
- **Extraction** → je transforme l'info
- **Automatisation** → j'agis proprement